Memory System

Microcomputer Memory

- Memory is an essential component of the microcomputer system.
- It stores binary instructions and datum for the microcomputer.
- The memory is the place where the computer holds current programs and data that are in use.
- None technology is optimal in satisfying the memory requirements for a computer system.
- Computer memory exhibits perhaps the widest range of type, technology, organization, performance and cost of any feature of a computer system.
- The memory unit that communicates directly with the CPU is called main memory.
- Devices that provide backup storage are called auxiliary memory or secondary memory.

Characteristics of memory systems

The memory system can be characterised with their Location, Capacity, Unit of transfer, Access method, Performance, Physical type, Physical characteristics, Organisation.

Location

- Processor memory: The memory like registers is included within the processor and termed as processor memory.
- Internal memory: It is often termed as main memory and resides within the CPU.
- External memory: It consists of peripheral storage devices such as disk and magnetic tape that are accessible to processor via i/o controllers.

Capacity

- Word size: Capacity is expressed in terms of words or bytes.
  - The natural unit of organisation
- Number of words: Common word lengths are 8, 16, 32 bits etc.
  - or Bytes

Unit of Transfer

- Internal: For internal memory, the unit of transfer is equal to the number of data lines into and out of the memory module.
- External: For external memory, they are transferred in block which is larger than a word.
- Addressable unit
  - Smallest location which can be uniquely addressed
  - Word internally
  - Cluster on Magnetic disks

Access Method

- Sequential access: In this access, it must start with beginning and read through a specific linear sequence. This means access time of data unit depends on position of records (unit of data) and previous location.
  - e.g. tape
- Direct Access: Individual blocks of records have unique address based on location. Access is accomplished by jumping (direct access) to general vicinity plus a sequential search to reach the final location.
  - e.g. disk
- Random access: The time to access a given location is independent of the sequence of prior accesses and is constant. Thus any location can be selected out randomly and directly addressed and accessed.
  - e.g. RAM
- Associative access: This is random access type of memory that enables one to make a comparison of desired bit locations within a word for a specified match, and to do this for all words simultaneously.
  - e.g. cache

Performance

- Access time: For random access memory, access time is the time it takes to perform a read or write operation i.e. time taken to address a memory plus to read / write from addressed memory location. Whereas for non-random access, it is the time needed to position read / write mechanism at desired location.
  - Time between presenting the address and getting the valid data
- Memory Cycle time: It is the total time that is required to store next memory access operation from the previous memory access operation.
  Memory cycle time = access time plus transient time (any additional time required before a second access can commence).
  - Time may be required for the memory to "recover" before next access
  - Cycle time is access + recovery
- Transfer Rate: This is the rate at which data can be transferred in and out of a memory unit.
  - Rate at which data can be moved
  - For random access, R = 1 / cycle time
  - For non-random access, Tn = Ta + N / R; where Tn – average time to read or write N bits, Ta – average access time, N – number of bits, R – Transfer rate in bits per second (bps).

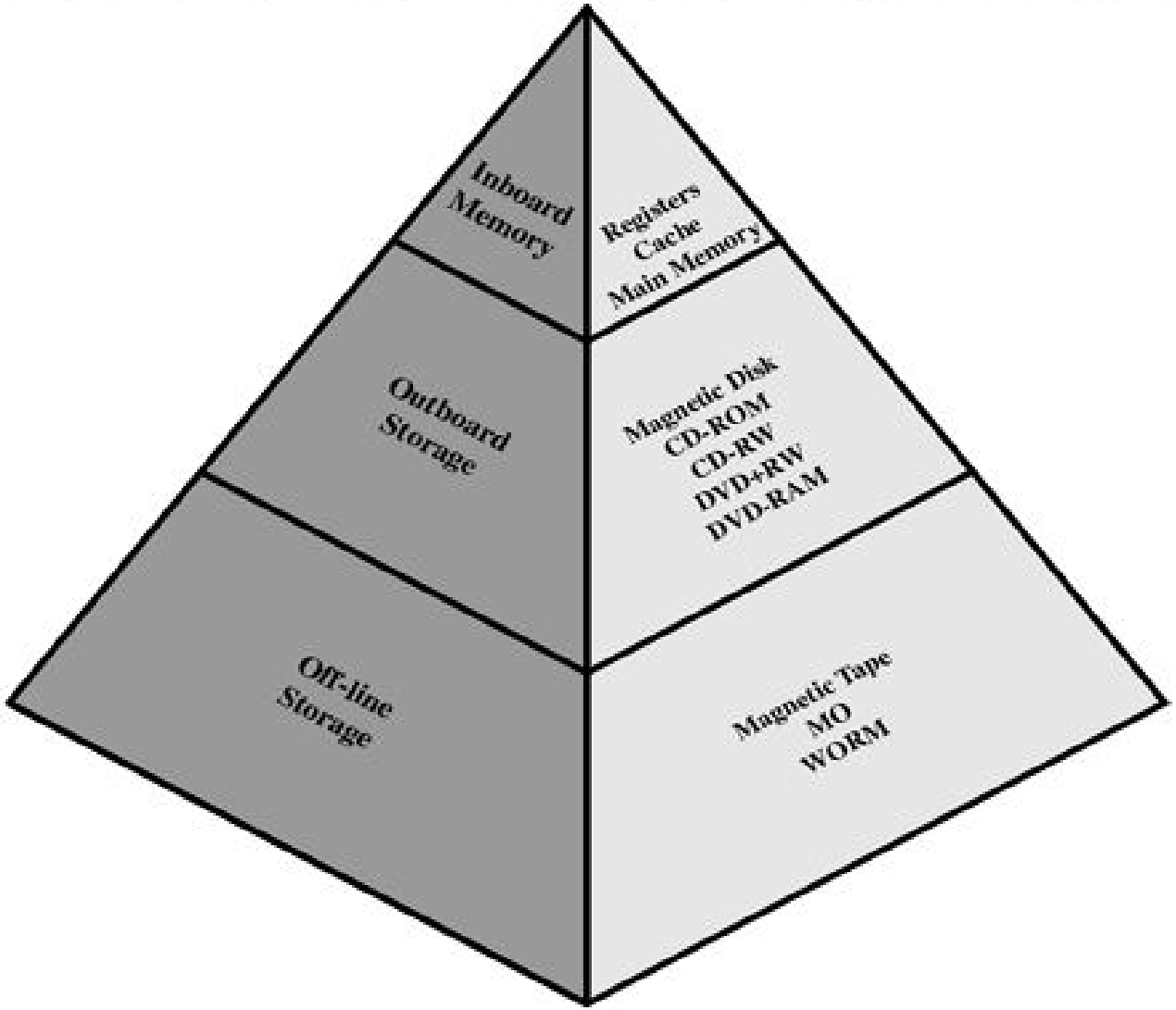
The Memory Hierarchy

- Capacity, cost and speed of different types of memory play a vital role while designing a memory system for computers.
- If the memory has larger capacity, more application will get space to run smoothly.
- It's better to have fastest memory as far as possible to achieve a greater performance. Moreover for the practical system, the cost should be reasonable.
- There is a tradeoff between these three characteristics cost, capacity and access time. One cannot achieve all these quantities in same memory module because
    - If capacity increases, access time increases (slower) and due to which cost per bit decreases.
    - If access time decreases (faster), capacity decreases and due to which cost per bit increases.
- The designer tries to increase capacity because cost per bit decreases and the more application program can be accommodated. But at the same time, access time increases and hence decreases the performance.

So the best idea will be to use memory hierarchy.

- Memory Hierarchy is to obtain the highest possible access speed while minimizing the total cost of the memory system.
- Not all accumulated information is needed by the CPU at the same time.
- Therefore, it is more economical to use low-cost storage devices to serve as a backup for storing the information that is not currently used by CPU
- The memory unit that directly communicate with CPU is called the *main memory*
- Devices that provide backup storage are called *auxiliary memory*
- The memory hierarchy system consists of all storage devices employed in a computer system from the slow by high-capacity auxiliary memory to a relatively faster main memory, to an even smaller and faster cache memory
- The main memory occupies a central position by being able to communicate directly with the CPU and with auxiliary memory devices through an I/O processor
- A special very-high-speed memory called **cache** is used to increase the speed of processing by making current programs and data available to the CPU at a rapid rate

- CPU logic is usually faster than main memory access time, with the result that processing speed is limited primarily by the speed of main memory
- The cache is used for storing segments of programs currently being executed in the CPU and temporary data frequently needed in the present calculations
- The memory hierarchy system consists of all storage devices employed in a computer system from slow but high capacity auxiliary memory to a relatively faster cache memory accessible to high speed processing logic. The figure below illustrates memory hierarchy.



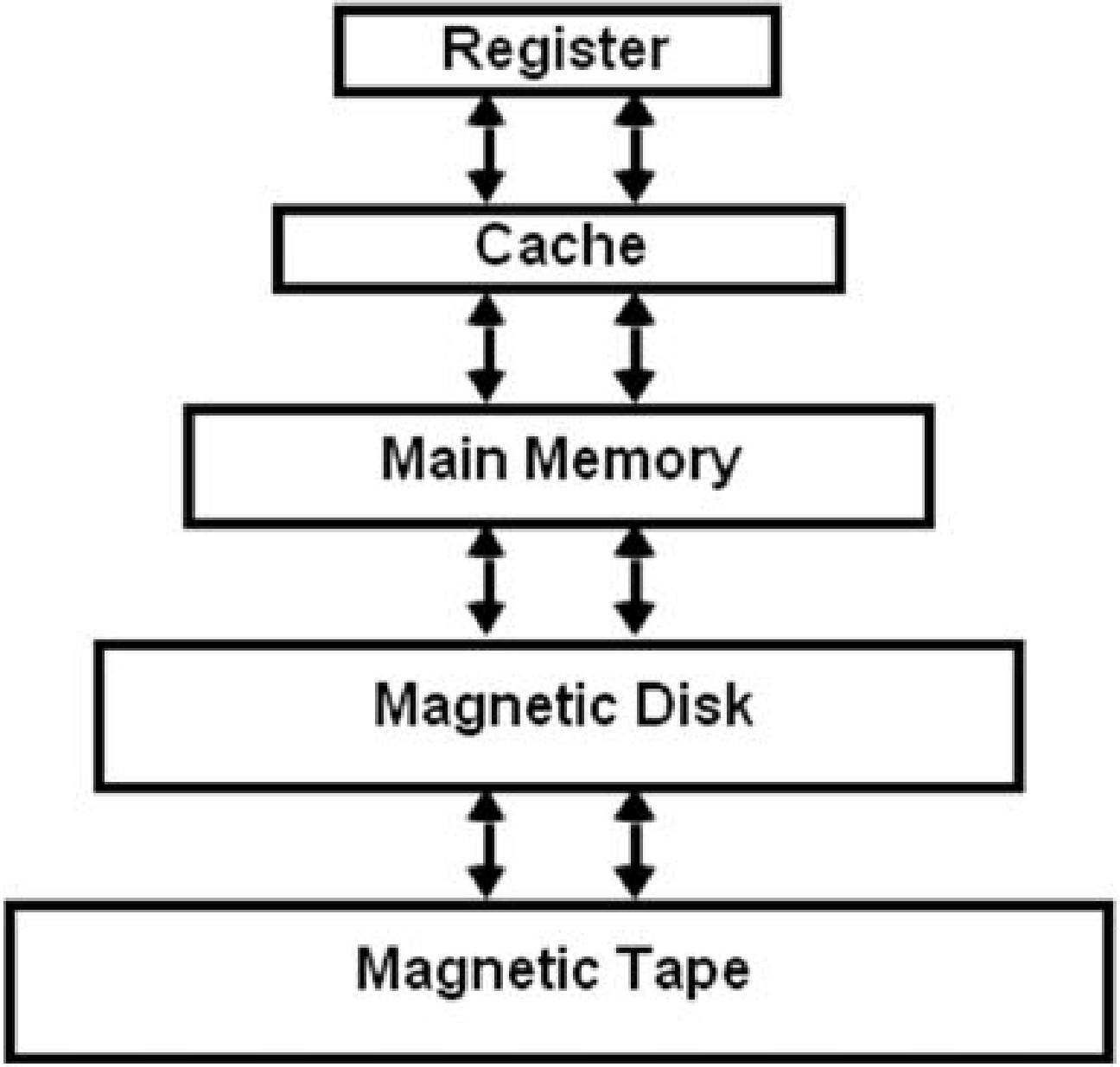


Fig: Memory Hierarchy

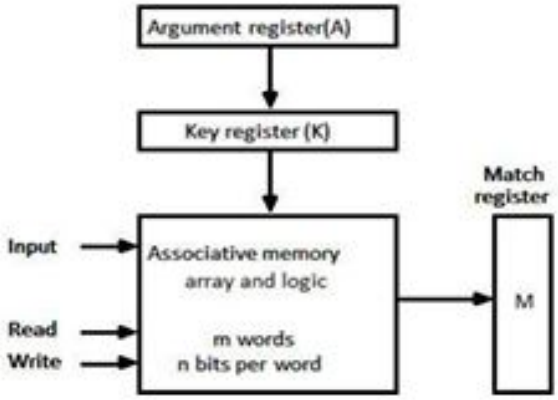
- As we go down in the hierarchy
  - Cost per bit decreases
  - Capacity of memory increases
  - Access time increases
  - Frequency of access of memory by processor also decreases.

Hierarchy List

- Registers
- L1 Cache
- L2 Cache
- Main memory
- Disk cache
- Disk
- Optical
- Tape

Associative Memory :Content Addressable Memory (CAM).

- The time required to find an item stored in memory can be reduced considerably if stored data can be identified for access by the content of the data itself rather than by an address.
- A memory unit accessed by content is called an associative memory or content addressable memory (CAM).
- This type of memory is accessed simultaneously and in parallel on the basis of data content rather than by specific address or location.
- The block diagram of an associative memory is shown in figure 9.3.



- It consists of a memory array and logic form words with n bits per word.
- The argument register A and key register K each have n bits, one for each bit of a word.
- The match register M has m bits, one for each memory word.
- Each word in memory is compared in parallel with the content of the argument register.
- The words that match the bits of the argument register set a corresponding bit in the match register.
- After the matching process, those bits in the match register that have been set indicate the fact that their corresponding words have been matched.
- Reading is accomplished by a sequential access to memory for those words whose corresponding bits in the match register have been set.

Hardware Organization

- The key register provides a mask for choosing a particular field or key in the argument word.
- The entire argument is compared with each memory word if the key register contains all 1's.

- Otherwise, only those bits in the argument that have 1st in their corresponding position of the key register are compared.
- Thus the key provides a mask or identifying piece of information which specifies how the reference to memory is made.
- To illustrate with a numerical example, suppose that the argument register A and the key register K have the bit configuration shown below.
- Only the three leftmost bits of A are compared with memory words because K has 1's in these position.

A	101 111100	
K	111 000000	
Word1	100 111100	no match
Word2	101 000001	match

- Word 2 matches the unmasked argument field because the three leftmost bits of the argument and the word are equal.

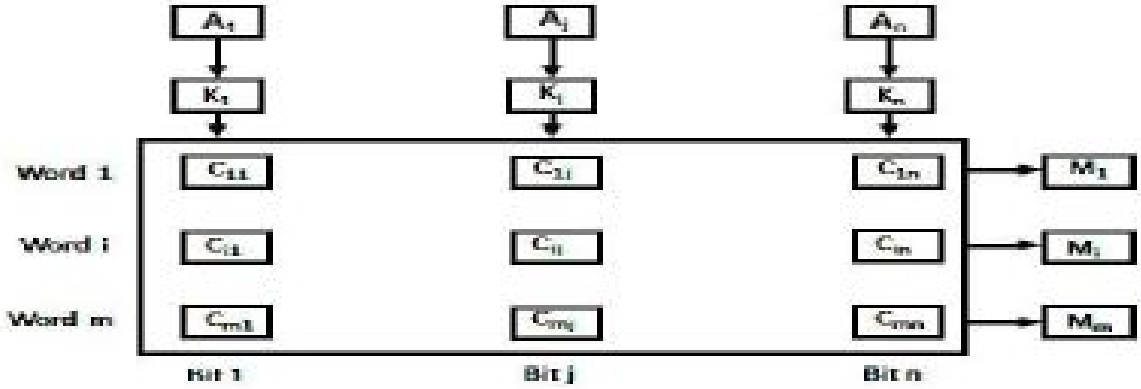


Figure 9.4: Associative memory of *m* word, n cells per word.

- The relation between the memory array and external registers in an associative memory is shown in figure 9.4.
- The cells in the array are marked by the letter C with two subscripts.
- The first subscript gives the word number and the second specifies the bit position in the word.
- Thus cell Cij is the cell for bit j in words i.
- A bit Aj in the argument register is compared with all the bits in column j of the array provided that Kj =1.
- This is done for all columns j = 1, 2... n.
- If a match occurs between all the unmasked bits of the argument and the bits in word i, the corresponding bit Mi in the match register is set to 1.
- If one or more unmasked bits of the argument and the word do not match, Mi is cleared to 0.

2.) Cache Memory:

- Small amount of fast memory
- Sits between normal main memory and CPU
- May be located on CPU chip or module

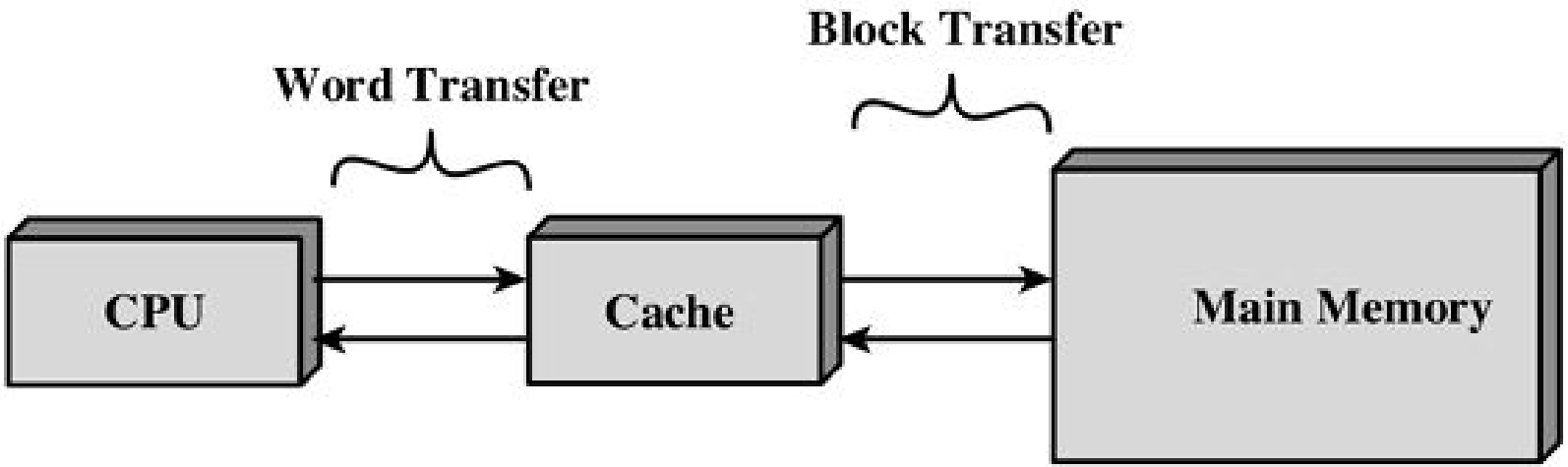


Figure: Single Cache

Cache memory is random access memory (RAM) that a computer microprocessor can access more quickly than it can access regular RAM. As the microprocessor processes data, it looks first in the cache memory and if it finds the data there (from a previous reading of data), it does not have to do the more time-consuming reading of data from larger memory.

Cache Read/ Write Policy:

Read policy: when a processor starts a read cycle the cache checks to see if that address is a cache hit.

Cache HIT: If the cache contains the memory location, then cache will respond to the read cycle & terminates the bus cycle.

Miss: If the cache does not contain the memory location, then main memory will respond to the processor & terminate the bus cycle. The cache will snarf (write itself) the data, so next time the processor request this data; it will be a cache hit.

Write policy: two common policies

i.)Write-Back Policy:

It acts like a buffer i.e. when the processor starts the write cycle the cache receives the data and terminates the cycle. The cache then writes the data back to main memory, when the data system bus is available & provides greatest performance but it increases complexity & cost.

ii.)Write-Through Policy:

The processor writes through the cache to main memory. This method is less complex and expensive but has lower performance.

- When the CPU needs to access memory, cache is examined. If the word is found in cache, it is read from the cache and if the word is not found in cache, main memory is accessed to read word. A block of word containing the one just accessed is then transferred from main memory to cache memory.

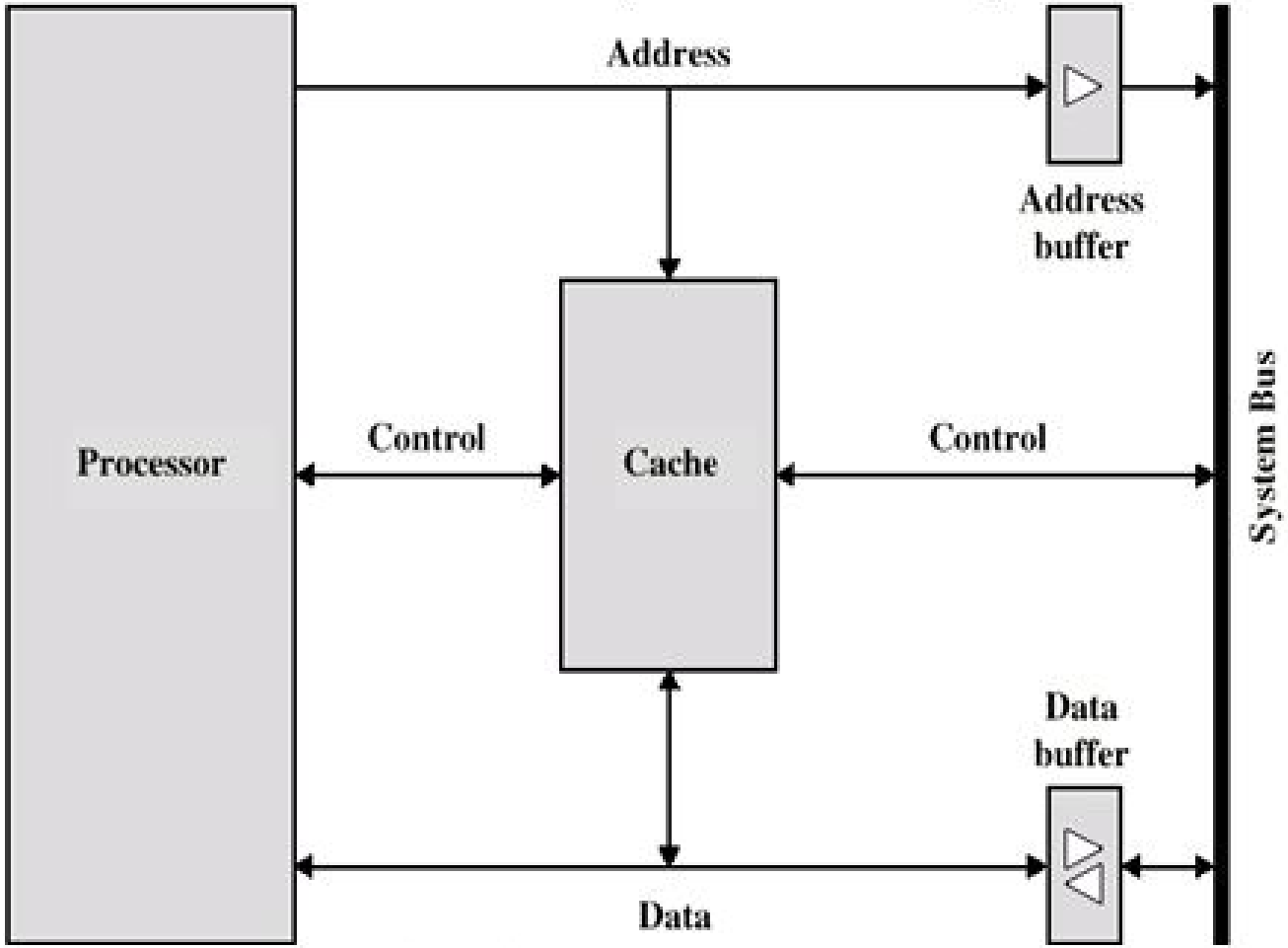


Fig: Typical Cache organization

- Cache connects to the processor via data control and address line. The data and address lines also attached to data and address buffer which attached to a system bus from which main memory is reached.

- When a cache hit occurs, the data and address buffers are disabled and the communication is only between processor and cache with no system bus traffic. When a cache miss occurs, the desired word is first read into the cache and then transferred from cache to processor. For later case, the cache is physically interposed between the processor and main memory for all data, address and control lines.

Cache Operation Overview

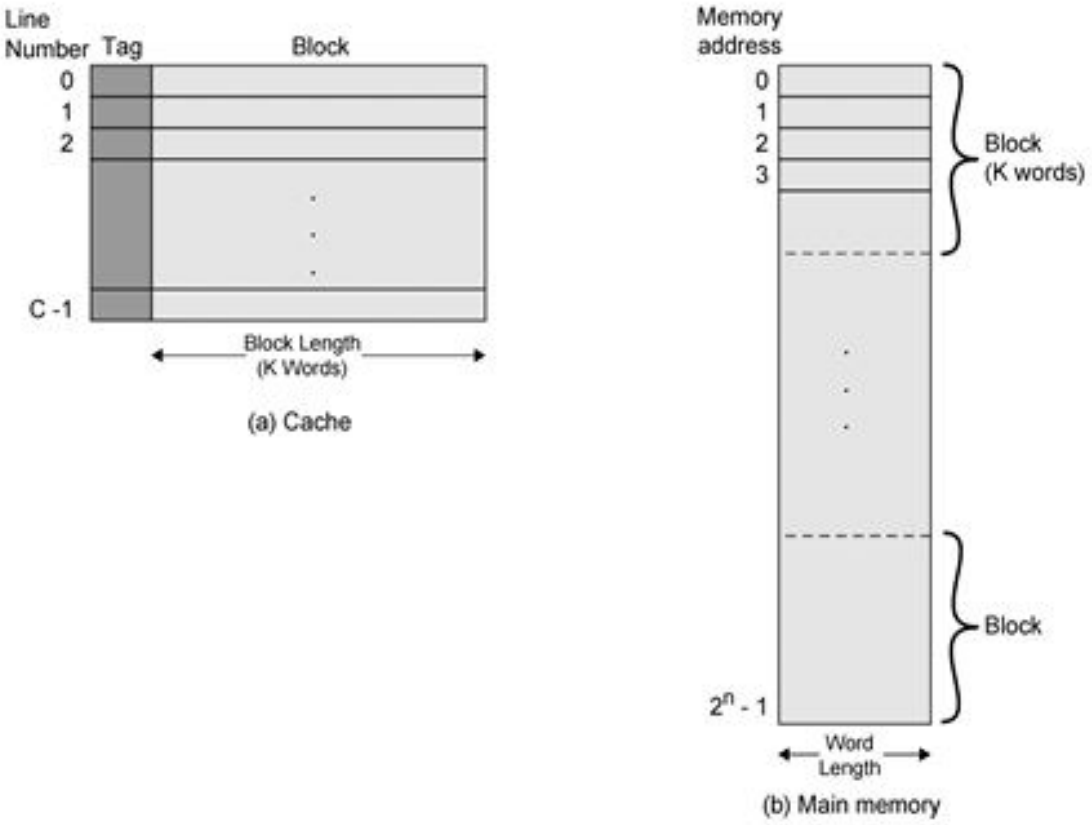


Fig: Cache memory / Main memory structure

- CPU generates the receive address (RA) of a word to be moved (read).
- Check a block containing RA is in cache.
- If present, get from cache (fast) and return.
- If not present, access and read required block from main memory to cache.
- Allocate cache line for this new found block.
- Load bock for cache and deliver word to CPU
- Cache includes tags to identify which block of main memory is in each cache slot

Locality of Reference

- The reference to memory at any given interval of time tends to be confined within a few localized area of memory. This property is called locality of reference. This is possible because the program loops and subroutine calls are encountered frequently. When program loop is executed, the CPU will execute same portion of program repeatedly. Similarly, when a subroutine is called, the CPU fetched starting address of subroutine and executes the subroutine program. Thus loops and subroutine localize reference to memory.
- This principle states that memory references tend to cluster over a long period of time, the clusters in use changes but over a short period of time, the processor is primarily working with fixed clusters of memory references.

Spatial Locality

- It refers to the tendency of execution to involve a number of memory locations that are clustered.
- It reflects tendency of a program to access data locations sequentially, such as when processing a table of data.

Temporal Locality

- It refers to the tendency for a processor to access memory locations that have been used frequently. For e.g. Iteration loops executes same set of instructions repeatedly.

6.6 Elements of Cache design

6.6.1 Cache size

- Size of the cache to be small enough so that the overall average cost per bit is close to that of main memory alone and large enough so that the overall average access time is close to that of the cache alone.
- The larger the cache, the larger the number of gates involved in addressing the cache.
- Large caches tend to be slightly slower than small ones – even when built with the same integrated circuit technology and put in the same place on chip and circuit board.
- The available chip and board also limits cache size.

6.6.2 Mapping function

- The transformation of data from main memory to cache memory is referred to as memory mapping process.
- Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines.
- There are three different types of mapping functions in common use and are direct, associative and set associative. All the three include following elements in each example.
    - The cache can hold 64 Kbytes
    - Data is transferred between main memory and the cache in blocks of 4 bytes each. This means that the cache is organized as 16Kbytes = 2^{14} lines of 4 bytes each.
    - The main memory consists of 16 Mbytes with each byte directly addressable by a 24 bit address (2^{24} = 16Mbytes). Thus, for mapping purposes, we can consider main memory to consist of 4Mbytes blocks of 4 bytes each.

Direct Mapping

- It is the simplex technique, maps each block of main memory into only one possible cache line i.e. a given main memory block can be placed in one and only one place on cache.
  i = j modulo m
  Where I = cache line number; j = main memory block number; m = number of lines in the cache
- The mapping function is easily implemented using the address. For purposes of cache access, each main memory address can be viewed as consisting of three fields.
- The least significant w bits identify a unique word or byte within a block of main memory. The remaining s bits specify one of the 2^s blocks of main memory.
- The cache logic interprets these s bits as a tag of (s-r) bits most significant position and a line field of r bits. The latter field identifies one of the m = 2^r lines of the cache.

Tag s-r	Line or Slot r	Word w
8	14	2

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
- Number of lines in cache = m = 2^r
- Size of tag = (s – r) bits

- 24 bit address
- 2 bit word identifier (4 byte block)
- 22 bit block identifier
- 8 bit tag (=22-14), 14 bit slot or line
- No two blocks in the same line have the same Tag field
- Check contents of cache by finding line and checking Tag

Cache line	**Main Memory blocks held**
0	0, m, 2m, 3m…2s-m
1	1,m+1, 2m+1…2s-m+1
m-1	m-1, 2m-1,3m-1…2s-1

Cache Line	**0**	**1**	**2**	**3**	**4**
Main Memory Block	0	1	2	3	4
	5	6	7	8	9
	10	11	12	13	14
	15	16	17	18	19
	20	21	22	23	24

Note that

- *all locations in a single block of memory have the same higher order bits (call them the block number), so the lower order bits can be used to find a particular word in the block.*
- *within those higher-order bits, their lower-order bits obey the modulo mapping given above (assuming that the number of cache lines is a power of 2), so they can be used to get the cache line for that block*
- *the remaining bits of the block number become a tag, stored with each cache line, and used to distinguish one block from another that could fit into that same cache*

line.

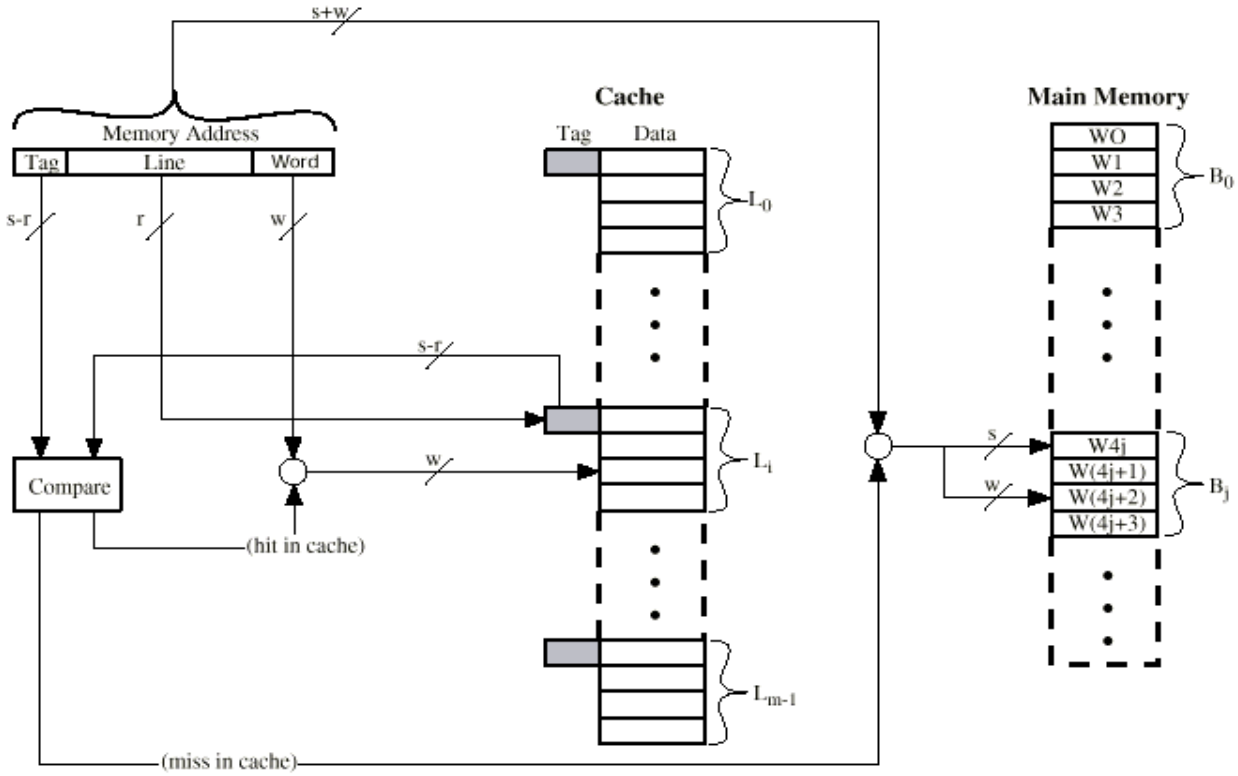
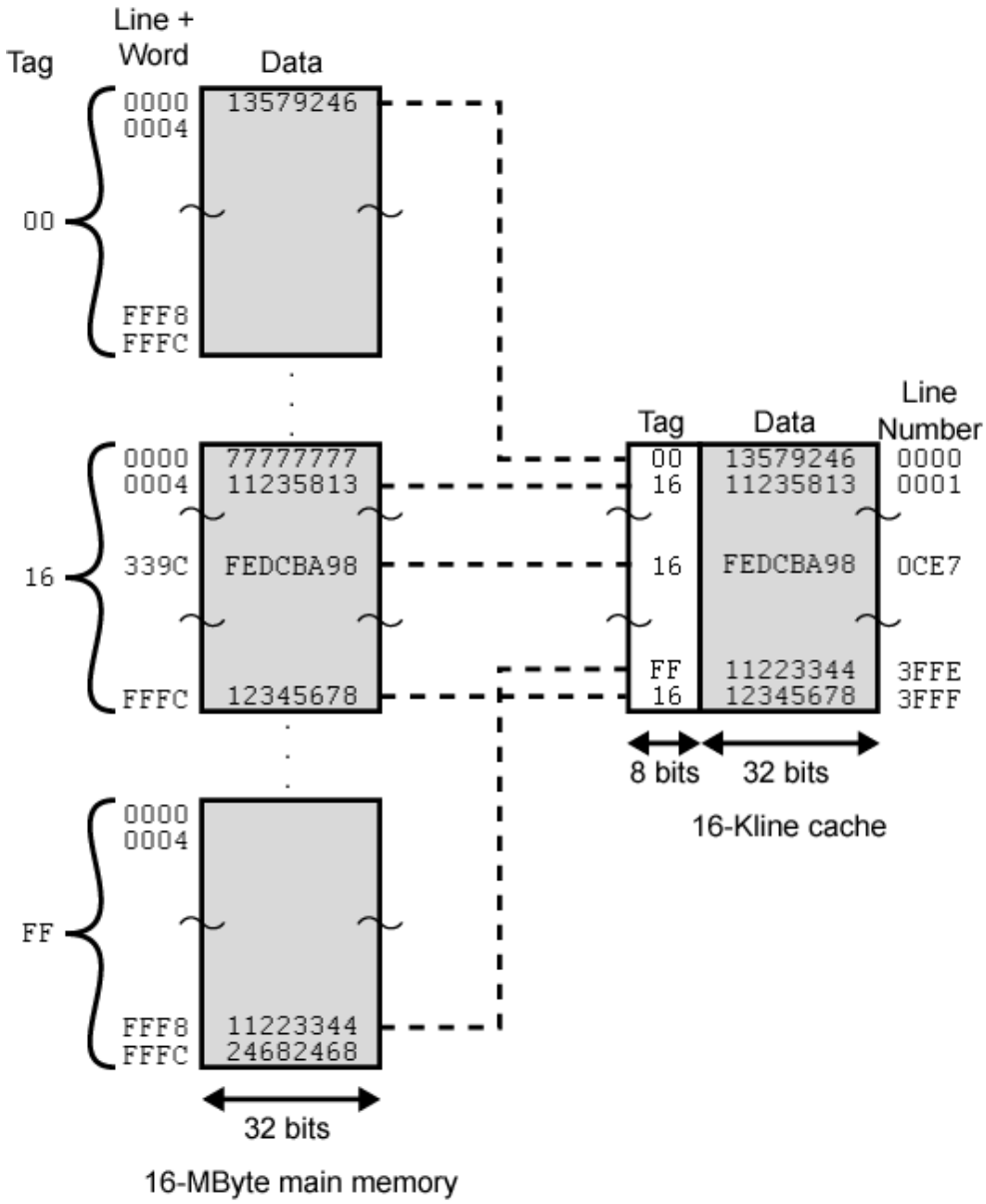


Fig: Direct mapping structure



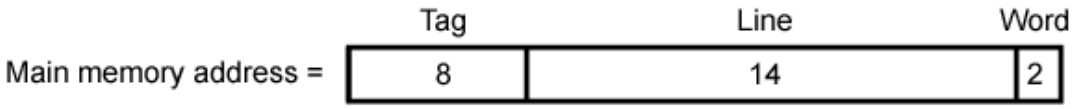


Fig: Direct mapping example

Pros and Cons

- Simple
- Inexpensive
- Fixed location for given block

- If a program accesses 2 blocks that map to the same line repeatedly, cache misses are very high

Associated Mapping

- It overcomes the disadvantage of direct mapping by permitting each main memory block to be loaded into any line of cache.
- Cache control logic interprets a memory address simply as a tag and a word field
- Tag uniquely identifies block of memory
- Cache control logic must simultaneously examine every line's tag for a match which requires fully associative memory
- very complex circuitry, complexity increases exponentially with size
- Cache searching gets expensive

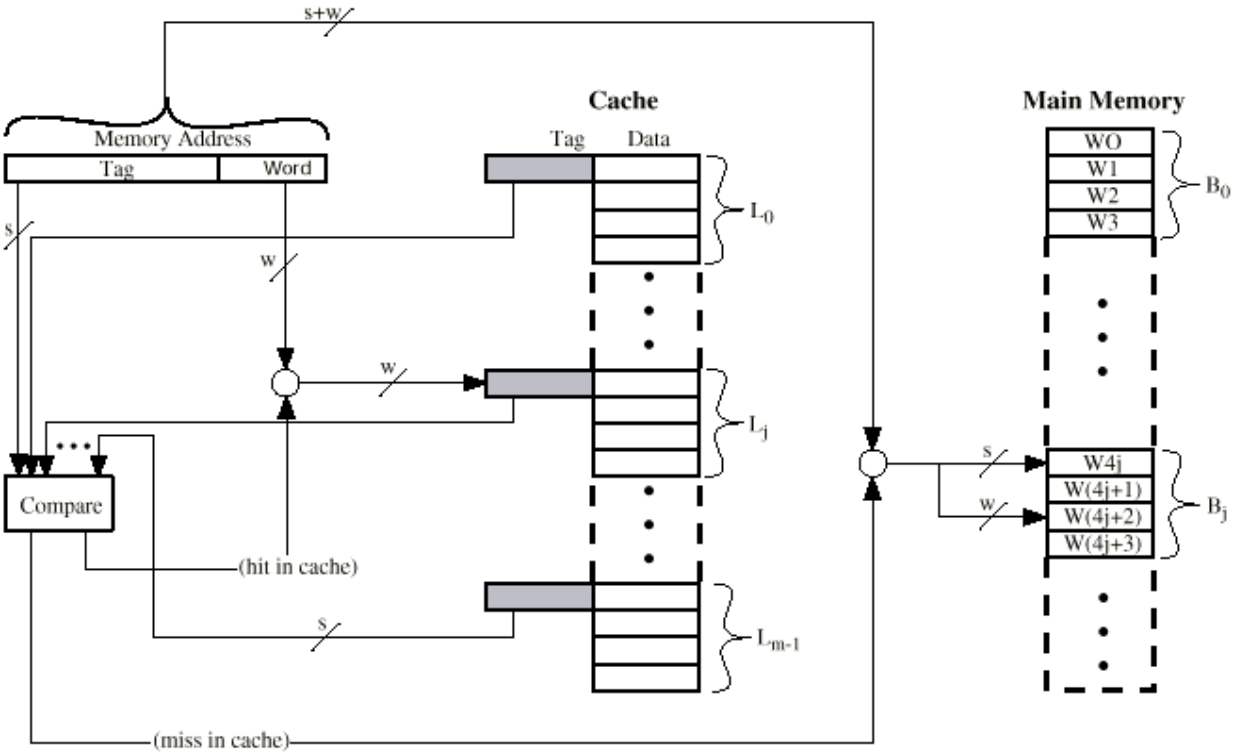
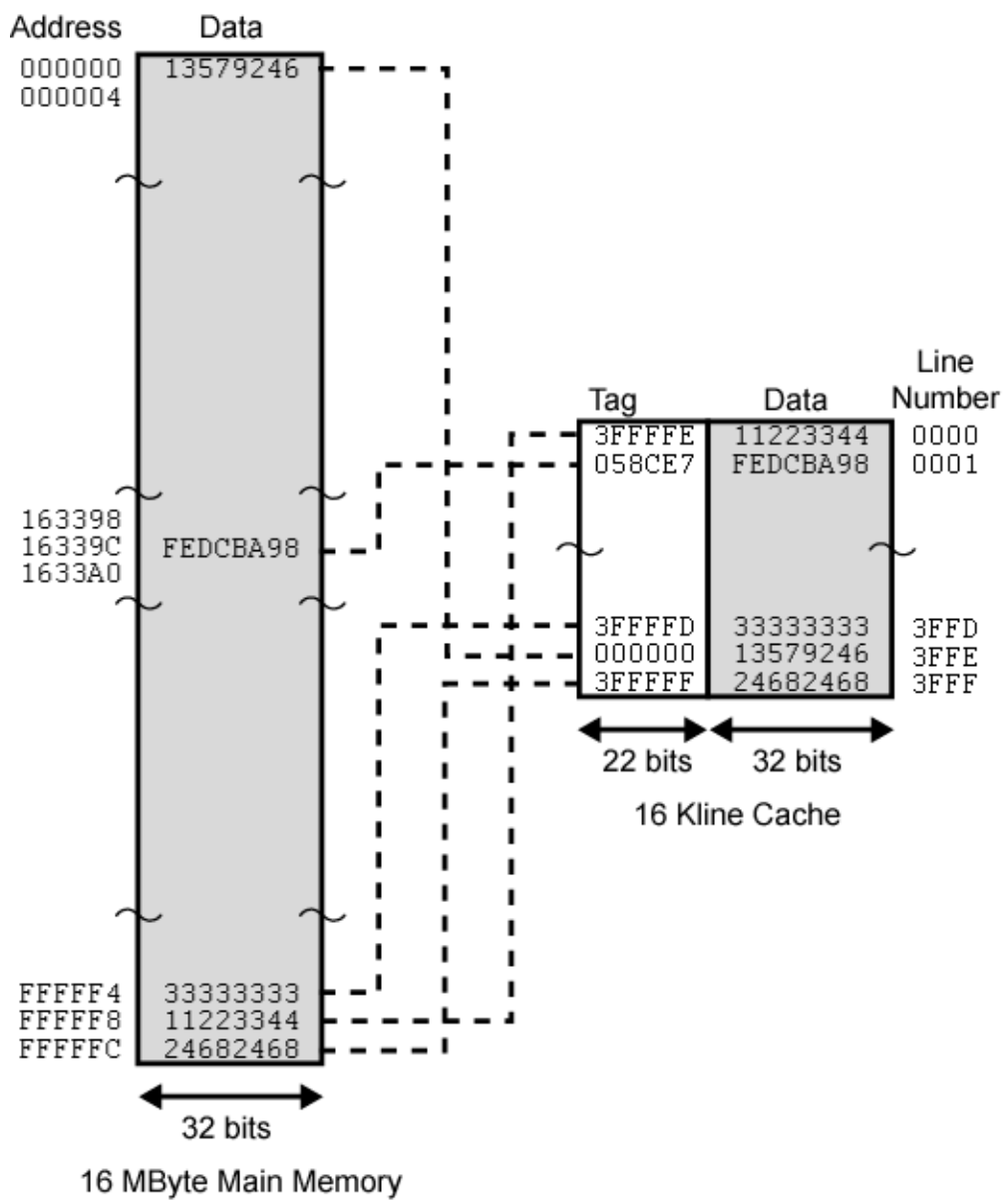


Fig: Associative structure

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes
- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = $2^{s+w}/2^w = 2^s$
- Number of lines in cache = undetermined, Size of tag = s bits



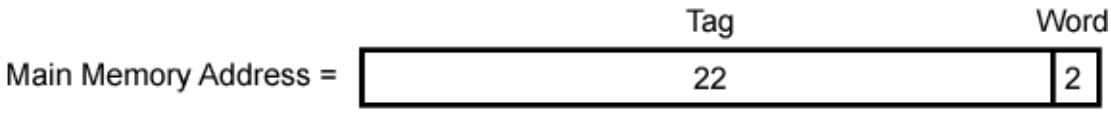


Fig: Associative mapping example

- 22 bit tag stored with each 32 bit block of data
- Compare tag field with tag entry in cache to check for hit
- Least significant 2 bits of address identify which 16 bit word is required from 32 bit data block
- e.g.

Address	Tag	Data	Cache line
FFFFFC	FFFFFC	24682468	3FFF

Set Associated Mapping

- It is a compromise between direct and associative mappings that exhibits the strength and reduces the disadvantages
- Cache is divided into v sets, each of which has k lines; number of cache lines = vk
  M = v X k
  I = j modulo v
  Where, i = cache set number; j = main memory block number; m = number of lines in the cache
- So a given block will map directly to a particular set, but can occupy any line in that set (associative mapping is used within the set)
- Cache control logic interprets a memory address simply as three fields tag, set and word. The d set bits specify one of $v = 2^d$ sets. Thus s bits of tag and set fields specify one of the 2^s block of main memory.
- The most common set associative mapping is 2 lines per set, and is called two-way set associative. It significantly improves hit ratio over direct mapping, and the associative hardware is not too expensive.

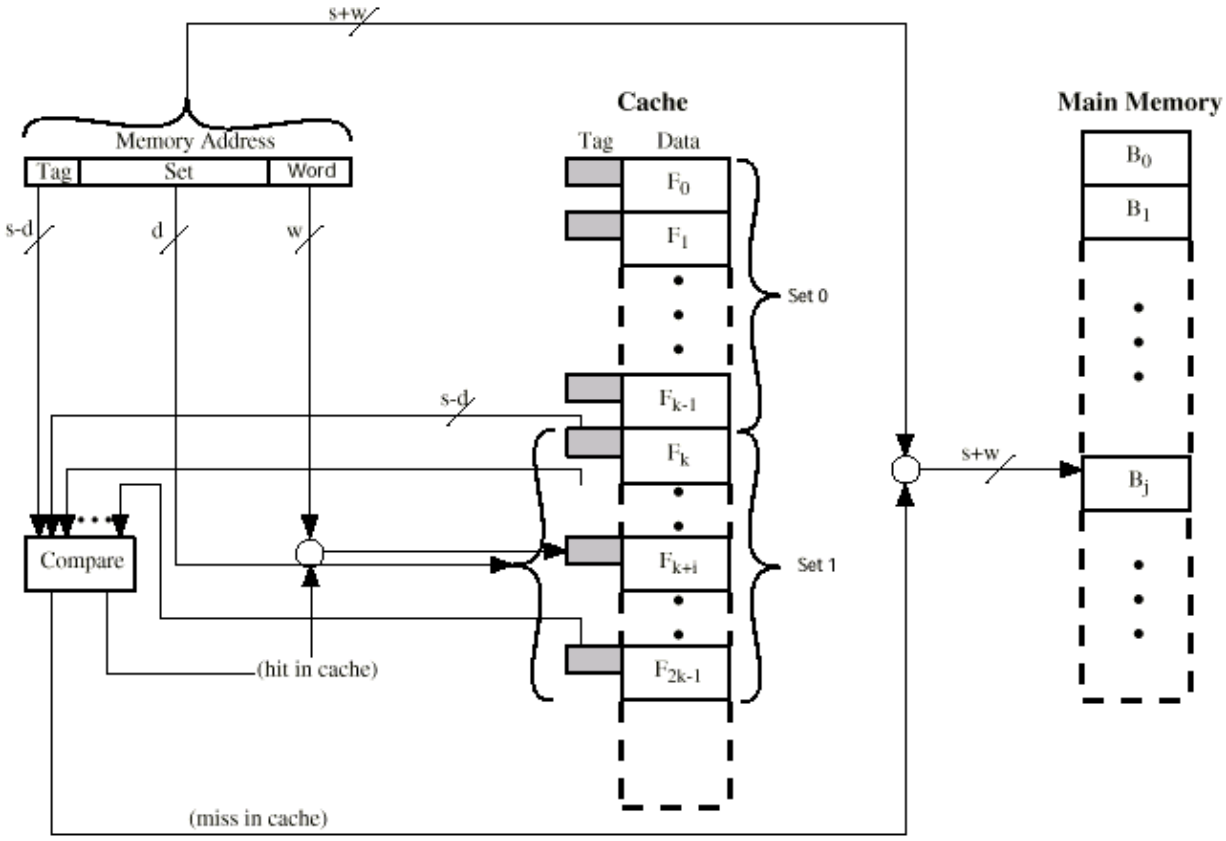
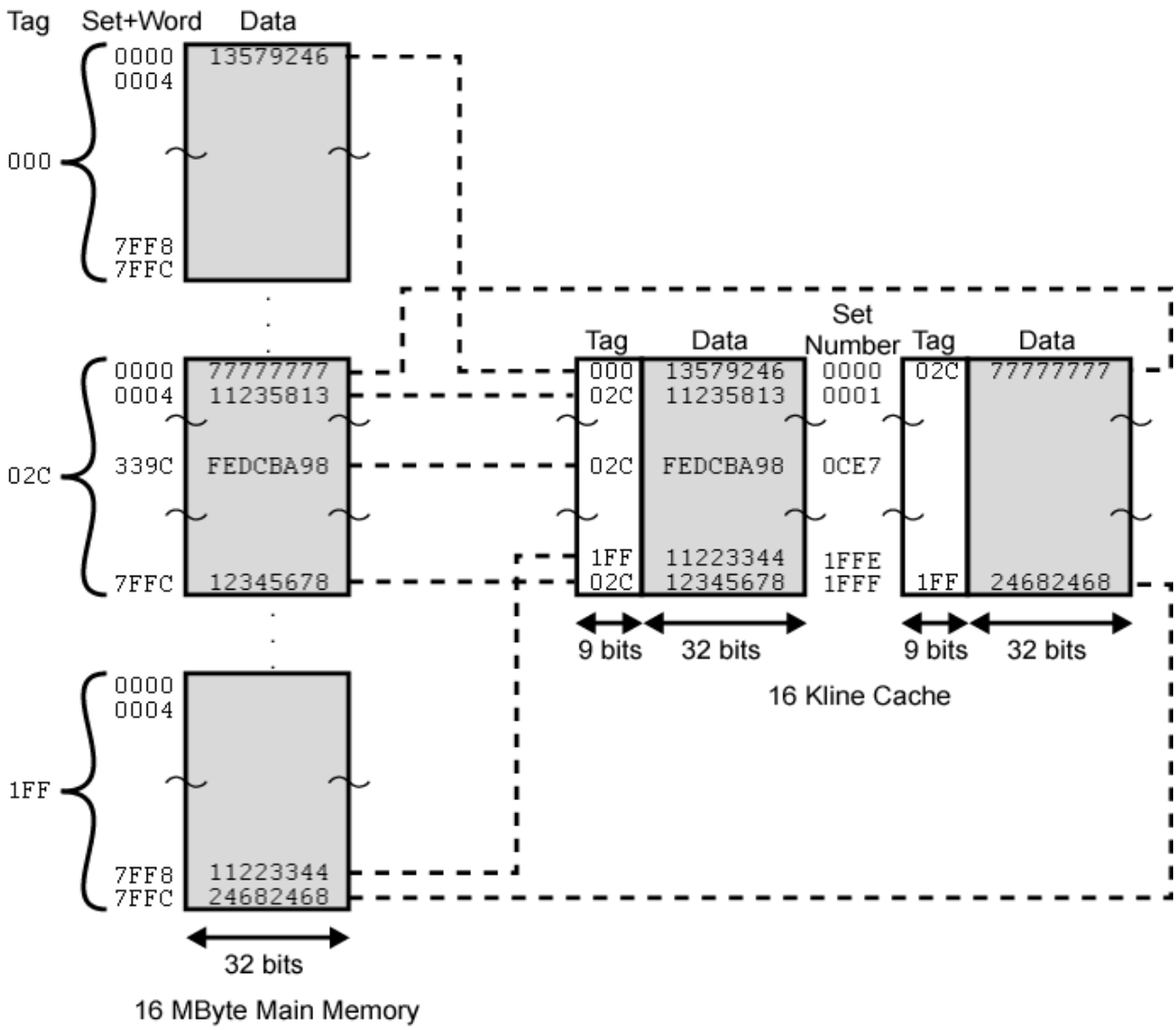


Fig: Set associative mapping structure

- Address length = (s + w) bits
- Number of addressable units = 2^{s+w} words or bytes

- Block size = line size = 2^w words or bytes
- Number of blocks in main memory = 2^d
- Number of lines in set = k
- Number of sets = v = 2^d
- Number of lines in cache = kv = k * 2^d
- Size of tag = (s – d) bits



	Tag	Set	Word
Main Memory Address =	9	13	2

Fig: Set associative mapping example

- 13 bit set number
- Block number in main memory is modulo 2^{13}

- 000000, 00A000, 00B000, 00C000 … map to same set
- Use set field to determine cache set to look in
- Compare tag field to see if we have a hit

- e.g

Address	Tag	Data	Set number
1FF 7FFC	1FF	12345678	1FFF
001 7FFC	001	11223344	1FFF

6.6.3 Replacement algorithm

- When all lines are occupied, bringing in a new block requires that an existing line be overwritten.

Direct mapping

- No choice possible with direct mapping
- Each block only maps to one line
- Replace that line

Associative and Set Associative mapping

- Algorithms must be implemented in hardware for speed
- Least Recently used (LRU)
  - replace that block in the set which has been in cache longest with no reference to it
  - Implementation: with 2-way set associative, have a USE bit for each line in a set. When a block is read into cache, use the line whose USE bit is set to 0, then set its USE bit to one and the other line's USE bit to 0.
  - Probably the most effective method
- First in first out (FIFO)
  - replace that block in the set which has been in the cache longest
  - Implementation: use a round-robin or circular buffer technique (keep up with which slot's "turn" is next
- Least-frequently-used (LFU)
  - replace that block in the set which has experienced the fewest references or hits
  - Implementation: associate a counter with each slot and increment when used
- Random
  - replace a random block in the set
  - Interesting because it is only slightly inferior to algorithms based on usage

6.6.4 Write policy

- When a line is to be replaced, must update the original copy of the line in main memory if any addressable unit in the line has been changed
- If a block has been altered in cache, it is necessary to write it back out to main memory before replacing it with another block (writes are about 15% of memory references)
- Must not overwrite a cache block unless main memory is up to date
- I/O modules may be able to read/write directly to memory
- Multiple CPU's may be attached to the same bus, each with their own cache

Write Through

- All write operations are made to main memory as well as to cache, so main memory is always valid
- Other CPU's monitor traffic to main memory to update their caches when needed
- This generates substantial memory traffic and may create a bottleneck
- Anytime a word in cache is changed, it is also changed in main memory
- Both copies always agree
- Generates lots of memory writes to main memory
- Multiple CPUs can monitor main memory traffic to keep local (to CPU) cache up to date
- Lots of traffic
- Slows down writes
- Remember bogus write through caches!

Write back

- When an update occurs, an UPDATE bit associated with that slot is set, so when the block is replaced it is written back first
- During a write, only change the contents of the cache
- Update main memory only when the cache line is to be replaced
- Causes "cache coherency" problems -- different values for the contents of an address are in the cache and the main memory
- Complex circuitry to avoid this problem
- Accesses by I/O modules must occur through the cache
- Multiple caches still can become invalidated, unless some cache coherency system is used. Such systems include:
    - Bus Watching with Write Through - other caches monitor memory writes by other caches (using write through) and invalidates their own cache line if a match
    - Hardware Transparency - additional hardware links multiple caches so that writes to one cache are made to the others
    - Non-cacheable Memory - only a portion of main memory is shared by more than one processor, and it is non-cacheable